

week4 연습과제

≡ 다중 선택	week4
≡ 다중 선택 1	

5.3 전이 학습

전이 학습(Transfer Learning)

: 아주 큰 데이터셋(ex. ImageNet)을 사용해 미리 훈련된 모델의 가중치를 가져와, 해결하려는 새로운 과제에 맞게 보정하여 사용하는 방법

- CNN 기반 딥러닝 모델을 처음부터 제대로 훈련하려면 많은 양의 데이터가 필요
- 현실적으로 충분히 큰 데이터셋을 확보하기 어려움 → **전이 학습**을 사용
- 비교적 적은 데이터로도 원하는 과제를 해결할 수 있음
- 아주 큰 데이터셋으로 미리 학습된 모델 → **사전 훈련된 모델(Pre-trained Model)**

전이 학습 방법

1. 특성 추출(Feature Extraction)
2. 미세 조정(Fine-tuning)

5.3.1 특성 추출 기법

특성 추출(Feature Extraction)

: 사전 훈련된 모델의 합성곱층을 그대로 사용하고, 마지막 완전연결층만 새로운 과제에 맞게 학습하는 방법

→ 사전 훈련된 모델의 합성곱층에서 이미 학습된 가중치는 **고정**

→ 추출된 특성을 새로운 **데이터 분류기(완전연결층)**에 입력하여 학습

구성

- **합성곱층**
 - 합성곱층 + 풀링층으로 구성
 - 사전 훈련된 네트워크에 새로운 데이터를 통과시켜 특성을 추출
- **데이터 분류기(완전연결층)**
 - 추출된 특성을 입력받아 최종적으로 클래스 분류
 - 새로운 과제에 맞게 학습

PyTorch에서 사전 훈련된 모델 사용

```
import torchvision.models as models

resnet18 = models.resnet18(pretrained=True)
```

`pretrained=True`

→ 사전 학습된 가중치를 사용

ResNet18

→ ImageNet 데이터셋으로 훈련된 사전 학습 모델

일부 파라미터 고정

```
for param in model.parameters():
    param.requires_grad = False
```

`requires_grad = False`

→ 해당 파라미터의 기울기를 계산하지 않음

→ 모델의 해당 부분을 **고정하고 학습하지 않음**

```
model.fc = nn.Linear(512, 2)
```

→ 기존 완전연결층을 새로운 분류 문제에 맞게 변경

→ `2`는 분류할 클래스가 2개라는 의미

```
for param in model.fc.parameters():
    param.requires_grad = True

optimizer = torch.optim.Adam(model.fc.parameters())
```

→ 새로 추가한 완전연결층만 학습

5.3.2 미세 조정 기법

미세 조정(Fine-tuning)

: 사전 훈련된 모델의 가중치를 새로운 목적에 맞게 **일부 또는 전체 재학습**하는 방법

→ 특성 추출보다 더 많은 계층의 가중치를 업데이트

📌 파라미터를 크게 변경하면 과적합이 발생할 수 있기 때문에 **필요한 부분만 미세하게 업데이트**하는 것이 중요

데이터셋 크기와 사전 훈련 모델의 유사성에 따른 전략

데이터셋	사전 훈련 모델과의 유사성	학습 방법
큼	작음	모델 전체 재학습

데이터셋	사전 훈련 모델과의 유사성	학습 방법
큼	큼	합성곱층 뒷부분 + 데이터 분류기 학습
작음	작음	합성곱층 일부 + 데이터 분류기 학습
작음	큼	데이터 분류기만 학습

→ 데이터가 적을수록 많은 계층을 재학습하면 과적합이 발생할 수 있음

5.4 설명 가능한 CNN

설명 가능한 CNN(Explainable CNN)

: 딥러닝의 처리 결과를 사람이 이해할 수 있는 방식으로 제시하는 방법

CNN은 내부에서 어떤 특징을 보고 판단했는지 직접 확인하기 어려운 **블랙박스**와 같음

→ 결과를 신뢰하기 어려울 수 있음

⇒ CNN의 처리 과정을 시각화하여

입력 이미지 → **각 계층에서 특성 추출** → **최종 분류** 과정을 확인

설명 가능한 CNN의 시각화 방법

1. 필터 시각화
2. 특성 맵 시각화

5.4.1 특성 맵 시각화

특성 맵(Feature Map)

: 입력 이미지 또는 다른 특성 맵에 필터를 적용하여 얻은 결과

→ 특정 입력 이미지에서 CNN이 어떤 특성을 감지하는지 확인할 수 있음

특성 맵 확인 과정

```
class LayerActivations:
    features = []

    def __init__(self, model, layer_num):
        self.hook = model[layer_num].register_forward_hook(self.hook_fn)

    def hook_fn(self, module, input, output):
        self.features = output.detach().numpy()

    def remove(self):
        self.hook.remove()
```

register_forward_hook()

→ 특정 계층의 **순전파 출력값을 가져오기 위한 hook** 등록

```
result = LayerActivations(model.features, 0)

model(img)
activations = result.features
```

→ 0 번째 계층의 출력 특성 맵을 저장

계층에 따른 특성 맵 변화

- **입력층에 가까운 계층**
→ 입력 이미지의 형태가 비교적 유지됨
- **출력층에 가까운 계층**
→ 원래 이미지의 형태는 찾아보기 어려워지고 **이미지의 특징들만 전달**

⇒ CNN이 깊어질수록 **구체적인 형태** → **추상적인 특징**으로 변환됨

5.5 그래프 합성곱 네트워크

그래프 합성곱 네트워크(GCN, Graph Convolutional Network)

: 이미지에 대한 합성곱을 **그래프 데이터로 확장한 알고리즘**

→ 그래프 형태의 데이터를 행렬 형태로 변환하여 딥러닝 알고리즘을 적용

5.5.1 그래프란?

그래프(Graph)

: 방향성이 있거나 없는 에지로 연결된 노드의 집합

- **노드(Node, Vertex)**
→ 그래프를 구성하는 각각의 원소
- **에지(Edge)**
→ 두 노드를 연결하는 관계

5.5.2 그래프 신경망

그래프 신경망(GNN, Graph Neural Network)

: 그래프 구조의 데이터를 처리하기 위한 신경망

그래프를 딥러닝에서 사용하기 위해

인접 행렬 + 특성 행렬로 표현

1) 인접 행렬(Adjacency Matrix)

: 노드 간의 연결 관계를 행렬로 표현

노드가 n 개라면 $\rightarrow n \times n$ 행렬

A_{ij}

$\rightarrow$ 노드 i 와 노드 j 의 관련성/연결 여부를 나타냄

$\Rightarrow$ 그래프의 구조와 연결 관계를 표현

2) 특성 행렬(Feature Matrix)

: 각 노드가 가지고 있는 특성을 행렬로 표현

$\rightarrow$ 인접 행렬만으로는 노드의 특성을 알기 어려움

$\rightarrow$ 각 입력 데이터에서 사용할 특성을 선택하여 표현

특성 행렬의 각 행

$\rightarrow$ 해당 특성에 대해 각 노드가 가지는 값

5.5.3 그래프 합성곱 네트워크

GCN(Graph Convolutional Network)

: 이미지에서 사용하는 합성곱 연산을 그래프 데이터에 적용한 알고리즘

그래프 합성곱층(Graph Convolutional Layer)

$\rightarrow$ 그래프 형태의 데이터를 행렬 형태로 변환

$\rightarrow$ 이후 딥러닝 알고리즘을 적용할 수 있도록 함

리드아웃(Readout)

: 특성 행렬을 하나의 벡터로 변환하는 함수

$\rightarrow$ 전체 노드의 특성 벡터에 대해 평균을 구하여

그래프 전체를 표현하는 하나의 벡터를 생성

활용 예시

- SNS의 관계 네트워크
- 학술 연구의 인용 네트워크
- 3D Mesh